Silent Numerical Failures in On-Device ML Converters: A Systematic Audit of FP16 Overflow in Apple Neural Engine Deployment

Ashutosh Kumar Singh
Independent Researcher
ashutoshkumarsingh0x@gmail.com
https://github.com/Ashutosh0x

June 2026

Abstract

Apple's Neural Engine (ANE) executes neural network operations natively in half-precision (`fp16`) arithmetic, where the maximum representable value is 65,504. We present a systematic audit of Apple's `coremltools` model converter—the standard pipeline for deploying PyTorch and TensorFlow models on iOS, iPadOS, and macOS—revealing **five operations** that silently produce incorrect results due to `fp16` overflow or underflow when executed on ANE. The affected operations—`softplus`, `mish`, `reduce_log_sum_exp`, `log_softmax`, and `logcumsumexp`—collectively impact every major model family deployed on Apple devices, including YOLO, BERT, ViT, MobileNetV3, and CTC-based speech recognizers.

We identify a consistent root cause: the converter emits mathematically correct but numerically naïve formulations that compute `exp(`x`)` on unbounded input, overflowing when $x > \ln(65{,}504) \approx 11.09$. For operations that reduce over C channels, the effective threshold drops to $\ln(65504/C)$; at vocabulary-sized reductions ($C \geq 1024$), this falls below 5—meaning virtually any non-trivial logit triggers overflow. For each vulnerable operation, we derive and implement the standard numerically stable decomposition using a max-shift transformation that bounds all intermediate `exp()` arguments to $(-\infty, 0]$, guaranteeing representability in `fp16`. We further document a *discrepancy pattern* where `coremltools`' own compile-time constant folding paths already use the stable formulas, but the runtime paths—which execute on ANE—do not.

All fixes have been submitted as pull requests to Apple's `coremltools` repository [Singh, 2026a,b,c] with accompanying regression tests using Conv2d-based models sized to ensure ANE routing. Our findings are independently corroborated by the Orion project [Kumaresan, 2026], which discovered the same overflow class while training LLMs directly on ANE. The critical nature of these vulnerabilities is further underscored by Apple's subsequent announcement of AFM 3 Core Advanced [Apple, 2026a]—a 20-billion-parameter sparse model executing entirely in `fp16` on ANE—and by the introduction of automatic stable decompositions in the Core AI framework [Apple, 2026b], the designated successor to Core ML.

1 Introduction

The deployment of neural networks on mobile and edge devices has become a critical capability for applications spanning computer vision, natural language processing, and speech recognition. Apple's Neural Engine (ANE), present in every iPhone since the A11 Bionic (2017) and every Mac since the M1 (2020), is the primary hardware accelerator for on-device machine learning inference on Apple platforms. As of 2026, over 2 billion active Apple devices contain an ANE.

The standard deployment pipeline for these devices uses Apple's `coremltools` converter [Apple, 2024], which translates models from frameworks such as PyTorch [Paszke et al., 2019] and TensorFlow [Abadi et al., 2016] into the Core ML format via an intermediate representation called MIL (Model Intermediate Language). By default, `coremltools` converts models to `fp16` precision (`compute_precision=FLOAT16`), enabling execution on ANE for maximum throughput.

However, `fp16` (IEEE 754 binary16) has a severely limited dynamic range: its maximum representable value is $2^{15} \times (1 + 1023/1024) = 65{,}504$. This means that $\exp(x)$ overflows for any $x > \ln(65{,}504) \approx 11.09$—a threshold easily exceeded by activation values in modern neural networks.

In this paper, we present a *systematic audit* of every operation in the `coremltools` PyTorch frontend converter that uses the exponential function, identifying which operations are vulnerable to `fp16` overflow on ANE. Our contributions are:

1. **A complete vulnerability census:** We audit all 12

operations in the converter that use `exp()`, classifying each as safe, vulnerable, or conditionally risky (Section 4).

2. **Identification of a discrepancy pattern:** We document cases where the converter's compile-time `value_inference` paths use numerically stable formulas, but the runtime code paths—which generate the actual ANE computation graph—use the naïve, unstable versions (Section 5).
3. **Proven fixes for all five vulnerable operations:** We derive and implement max-shift stable decompositions for each affected operation, with formal correctness arguments and regression tests (Section 6).
4. **Quantitative analysis of failure boundaries:** We characterize the exact input thresholds at which each operation transitions from correct to incorrect output in `fp16` (Section 7).
5. **Independent corroboration:** We connect our findings to the Orion project [Kumaresan, 2026], which independently discovered the same `fp16` overflow class in ANE training, and to reverse-engineering efforts that confirmed ANE's `fp16`-only constraint (Section 9).

2 Background

2.1 IEEE 754 Half-Precision (FP16)

The IEEE 754 binary16 format uses 1 sign bit, 5 exponent bits, and 10 mantissa bits (plus 1 implicit leading bit), providing:

- **Maximum value:** 65,504
- **Minimum positive normal:** $\approx 6.10 \times 10^{-5}$
- **Minimum positive subnormal:** $\approx 5.96 \times 10^{-8}$
- **Machine epsilon:** $2^{-10} \approx 9.77 \times 10^{-4}$
- **Decimal precision:** ∼3.31 significant digits

The critical threshold for the exponential function is:

$$\exp(x) > 65{,}504 \iff x > \ln(65{,}504) \approx 11.0896 \quad (1)$$

For values exceeding this threshold, $\exp(x)$ returns $+\infty$ in `fp16`, which then propagates through subsequent arithmetic as ∞ or NaN. Conversely, $\exp(x)$ underflows to 0 for $x < \ln(5.96 \times 10^{-8}) \approx -16.64$, which can cause $\log(0) = -\infty$ in downstream operations.

2.2 Apple Neural Engine Architecture

The Apple Neural Engine is a fixed-function neural network accelerator integrated into Apple's system-on-chip designs. Key characteristics relevant to numerical precision:

1. **Native `fp16` execution:** All arithmetic on ANE is performed in `fp16`. Unlike the GPU, which can fall back to `fp32` for individual operations, ANE has no `fp32` fallback path [Singh, 2025].
2. **No runtime overflow detection:** When an intermediate value overflows to ∞, ANE does not raise an exception or flag—the computation continues silently with corrupted values.
3. **Quantized operations dequantize to `fp16`:** Reverse-engineering of the M4 Neural Engine has revealed that even INT8 quantized operations are internally dequantized to `fp16` before computation [Singh, 2025], meaning `fp16` overflow bugs affect quantized deployments as well.

2.3 The coremltools Converter Pipeline

The `coremltools` converter translates trained models into the Core ML format through a multi-stage pipeline:

1. **Frontend parsing:** Framework-specific frontends (PyTorch, TensorFlow) traverse the model graph and emit MIL operations.
2. **MIL optimization:** Graph-level optimizations including constant folding, dead code elimination, and operator fusion.
3. **Backend lowering:** MIL operations are lowered to the target backend (`mlprogram` for ANE).

The PyTorch frontend converter resides in `converters/mil/frontend/torch/ops.py`, a ∼15,000-line file containing converter functions for ∼400 PyTorch operations. Each converter function takes a PyTorch operation node and emits equivalent MIL operations using builder calls such as `mb.exp()`, `mb.log()`, and `mb.add()`.

Each MIL operation definition includes a `value_inference()` method used during compile-time constant folding. This method operates in `fp32` or float64 and—as we will show—sometimes uses a different, numerically stable formula than the runtime path that generates the actual ANE computation.

3 Audit Methodology

Our audit systematically identifies `fp16`-unsafe operations in the `coremltools` converter through a three-stage process:

Stage 1: Exponential Call Census. We performed a complete search for all uses of `mb.exp()` in the PyTorch frontend converter (`ops.py`) and all uses of `np.exp()` in the MIL operation definitions (`converters/mil/mil/ops/defs/`). This identified every code path where the exponential function is applied to model activations at runtime.

Stage 2: Input Bound Analysis. For each `exp()` call site, we analyzed whether the argument is bounded:

- **Bounded below zero:** If $x \leq 0$ is guaranteed (*e.g.*, after a negation or clamping), then $\exp(x) \in (0, 1]$ and overflow is impossible. Operations like `elu` (which computes $\exp(x)$ only for $x < 0$) fall into this category.
- **Bounded by prior max-shift:** If the code subtracts $\max(x)$ before computing $\exp(x - \max(x))$, the argument is ≤ 0. Operations like `softmax` already do this.
- **Unbounded:** If $\exp(x)$ is applied to raw input with no bounding transformation, the operation is vulnerable.

Stage 3: Discrepancy Detection. For each vulnerable operation, we compared the runtime code path (the converter function in `ops.py`) against the `value_inference()` method in the corresponding MIL operation definition. This revealed cases where the compile-time path uses a stable formula but the runtime path does not.

4 Vulnerability Analysis

Table 1 presents the complete results of our audit across all 12 operations that use `exp()`.

4.1 Vulnerable Operations: Detailed Analysis

4.1.1 Softplus ($\log(1 + \exp(x))$)

The softplus activation is defined as $\text{softplus}(x) = \log(1 + \exp(x))$. The converter emits this as:

```
exp = mb.exp(x=x)
add = mb.add(x=exp, y=1.0)
res = mb.log(x=add)
```

For $x > 11.09$, `mb.exp(x)` returns $+\infty$ in `fp16`, producing $\log(1 + \infty) = \log(\infty) = +\infty$. However, ANE exhibits a more severe failure mode: the output collapses to **exactly 0.0** rather than $+\infty$, indicating hardware-specific overflow handling distinct from IEEE 754 semantics.

The overflow boundary in practice is $x \approx 10.4$ rather than 11.09, because the $1 + \exp(x)$ addition can lose precision before the explicit overflow threshold is reached.

Impact: Softplus is used in YOLOv5/v7/v8 detection heads, attention mechanisms, and variational autoencoders. The Mish activation ($x \cdot \tanh(\text{softplus}(x))$) inherits this bug, outputting 0 instead of $\approx x$ for $x > 10.4$.

4.1.2 Reduce Log-Sum-Exp ($\log \sum \exp(x_i)$)

The `reduce_log_sum_exp` operation computes $\log \sum_{i=1}^{C} \exp(x_i)$ over a reduction axis. The converter emits:

```
exp = mb.exp(x=x)
sum = mb.reduce_sum(x=exp, axes=[axis])
res = mb.log(x=sum)
```

For a reduction over C channels, the sum $\sum \exp(x_i)$ can reach $C \cdot \exp(x_{\max})$, which overflows when $x_{\max} > \ln(65504/C)$. For $C = 32$, this threshold is ≈ 7.63—well within the normal range of neural network activations.

4.1.3 Log-Softmax ($\log(\textbf{softmax}(x))$)

Unlike the previous operations, `log_softmax` fails via *underflow* rather than overflow. The converter computes:

```
res = mb.softmax(x=x, axis=axis)
res = mb.log(x=res)
```

While `softmax` internally uses max-shift stabilization, the resulting probabilities for non-dominant classes can be extremely small. In `fp16`, any probability below $\sim 6.10 \times 10^{-5}$ (the minimum positive normal) underflows to 0. Then $\log(0) = -\infty$, silently corrupting the output.

Consider a classification input $\mathbf{x} = [0, 0, 0, 50, 0, 0, 0, 0]$. The softmax probability for non-dominant classes is $e^{-50} \approx 1.93 \times 10^{-22}$, which underflows to 0 in `fp16`. The correct log-softmax value is -50, but the converter produces $-\infty$.

Impact: Every classification model using `nn.LogSoftmax`, `F.log_softmax`, or `F.cross_entropy`—which includes BERT, GPT, ViT, ResNet, and essentially every model that computes cross-entropy loss or log-probability outputs.

4.1.4 Log-Cumulative-Sum-Exp

The `logcumsumexp` operation computes the logarithm of the cumulative sum of exponentials along an axis: $y_j = \log \sum_{i=1}^{j} \exp(x_i)$. The converter emits:

```
exp = mb.exp(x=x)
cumsumexp = mb.cumsum(x=exp, axis=dim)
res = mb.log(x=cumsumexp)
```

This applies $\exp(x)$ to raw input with zero stabilization. For any $x > 11.09$, this overflows to $+\infty$ in `fp16`, and the cumulative sum propagates the infinity to all subsequent positions.

Impact: CTC decoders (speech recognition, OCR), autoregressive attention mechanisms, and sequential probability models.

5 The Value-Inference Discrepancy

A revealing finding of our audit is that `coremltools`' *own codebase already contains the numerically stable formulas*—but only in the compile-time constant folding paths, not in the runtime paths that generate the actual ANE computation.

Each MIL operation has a `value_inference()` method used during graph optimization to evaluate operations on constant inputs. For `softplus`, the `value_inference` at line 471 of `activation.py` computes:

Table 1: Complete audit of `exp()`-based operations in the `coremltools` PyTorch frontend converter. "Overflow Threshold" indicates the input value at which `fp16` overflow first occurs. "Discrepancy" indicates whether the `value_inference` path uses a different (stable) formula.

Operation	Naïve Formula	Safe?	Threshold	Discr.?	Status
`softplus`	$\log(1+e^x)$	No	$x>10.4$	Yes	Fixed (#2725)
`mish`	$x\tanh(\text{sp}(x))$	No	$x>10.4$	Yes	Fixed (#2725)
`reduce_lse`	$\log(\sum e^{x_i})$	No	$x>7.63$[a]	Yes	Fixed (#2726)
`log_softmax`	$\log(\text{sm}(x))$	No	$p<$6e-5[b]	No[c]	Fixed (#2727)
`logcumsumexp`	$\log(\text{cs}(e^x))$	No	$x>11.09$	No	Fixed (#2727)
`softmax`	$e^{x_i}/\sum e^{x_j}$	Yes	—	—	Safe (max-shift)
`elu`	$\alpha(e^x-1)$, $x<0$	Yes	—	—	Safe ($x<0$)
`selu`	uses `elu`	Yes	—	—	Safe
`gelu`	erf-based	Yes	—	—	Safe
`sp_parametric`	$\beta^{-1}\log(1+e^{\beta x})$	Risky	$\beta x>11.09$	Yes	Unfixed
`log_sigmoid`	$\log(\sigma(x))$	Risky	$x \ll 0$	—	Unfixed[d]
`expm1`	e^x-1	Risky	$x>11.09$	—	Unfixed

[a] C=32 channels: $\ln(65504/C)\approx 7.63$. [b] Underflow, not overflow: softmax probabilities below ∼6e-5 round to 0; $\log(0)=-\infty$. [c] No MIL-level op; the converter decomposes directly. [d] No registered converter; traces as $\log(\sigma(x))$.

```
# In value_inference (fp64, stable):
np.log(1 + np.exp(-np.abs(x)))
    + np.maximum(x, 0)
```

This is the textbook stable formula. But the *runtime* path in `ops.py` emits the naïve $\log(1+\exp(x))$—which overflows in `fp16`.

Similarly, for `reduce_log_sum_exp`, the `get_operator()` method in `reduction.py` computes:

```
# In get_operator (stable):
max_val = np.max(values, axis=axes)
exp_shifted = np.exp(values - max_val)
log_sum = np.log(np.sum(exp_shifted))
result = max_val + log_sum
```

Again, the stable formula with max-shift. But the runtime converter emits $\log(\sum\exp(x_i))$ without any stabilization.

This discrepancy pattern—*the code knows the right formula and uses it in one context, but fails to apply it where it actually matters*—suggests these are not oversights in mathematical knowledge but gaps in the engineering of the converter pipeline. The `value_inference` paths operate in float64 where overflow is not a concern; the runtime paths were likely written assuming similar numeric behavior but execute in `fp16` on ANE.

Table 2 summarizes the discrepancy for each vulnerable operation.

Table 2: Value-inference vs. runtime formula discrepancy.

Operation	Inference (stable)	Runtime (naïve)
`softplus`	$\max(x,0)+\log(1+e^{-\lvert x\rvert})$	$\log(1+e^x)$
`logsumexp`	$m+\log\sum e^{x_i-m}$	$\log\sum e^{x_i}$
`log_softmax`	N/A[a]	$\log(\text{softmax}(x))$
`logcumsumexp`	N/A[a]	$\log(\text{cumsum}(e^x))$

[a] No MIL-level op exists; the PyTorch converter decomposes these directly.

6 Stable Decompositions

All five fixes follow the same principle: **subtract the maximum before computing** exp(), ensuring all arguments are ≤ 0 and all intermediate exp() values are in $(0,1]$. The maximum is added back after log() to preserve mathematical equivalence.

6.1 General Max-Shift Identity

The fundamental identity underlying all fixes is:

$$\log\sum_{i=1}^{n}\exp(x_i) = \max(\mathbf{x}) + \log\sum_{i=1}^{n}\exp(x_i-\max(\mathbf{x})) \quad (2)$$

Proof. $\log\sum\exp(x_i) = \log\sum\exp(x_i-m+m) = \log\left[e^m\sum\exp(x_i-m)\right] = m+\log\sum\exp(x_i-m)$, where $m=\max(\mathbf{x})$.

Since $x_i-m\le 0$ for all i, we have $\exp(x_i-m)\in(0,1]$, which is always representable in `fp16`. □

6.2 Fix 1: Softplus

$$\text{softplus}(x) = \max(x, 0) + \log(1 + \exp(-|x|)) \quad (3)$$

Since $-|x| \leq 0$, we have $\exp(-|x|) \in (0, 1]$. The $\log(1+\cdot)$ term is in $[0, \log 2]$.

Proof of equivalence:

$$\begin{aligned}
\text{Case } x \geq 0: \quad & \max(x, 0) + \log(1 + e^{-x}) \\
& = x + \log(1 + e^{-x}) \\
& = x + \log \frac{e^x + 1}{e^x} \\
& = x + \log(1 + e^x) - x = \log(1 + e^x) \checkmark \\
\text{Case } x < 0: \quad & \max(x, 0) + \log(1 + e^x) \\
& = 0 + \log(1 + e^x) = \log(1 + e^x) \checkmark
\end{aligned}$$

6.3 Fix 2: Mish

$\text{mish}(x) = x \cdot \tanh(\text{softplus}(x))$. Applying the stable softplus from Equation (3) directly fixes mish with no additional changes.

6.4 Fix 3: Reduce-Log-Sum-Exp

Direct application of Equation (2):

$$\text{logsumexp}(\mathbf{x}) = m + \log \sum_i \exp(x_i - m) \quad (4)$$

where $m = \max(\mathbf{x})$ over the reduction axis.

6.5 Fix 4: Log-Softmax

$$\text{log\_softmax}(\mathbf{x})_i = (x_i - m) - \log \sum_j \exp(x_j - m) \quad (5)$$

Proof of equivalence:

$$\begin{aligned}
\log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right) & = x_i - \log \sum_j \exp(x_j) \\
& = x_i - m - \log \sum_j \exp(x_j - m) \\
& = (x_i - m) - \log \sum_j \exp(x_j - m)
\end{aligned}$$

This avoids computing the intermediate softmax probabilities, which underflow in `fp16`. All $\exp(x_j - m) \in (0, 1]$.

6.6 Fix 5: Log-Cumulative-Sum-Exp

$$\text{logcumsumexp}(\mathbf{x})_j = m + \log \sum_{i=1}^{j} \exp(x_i - m) \quad (6)$$

where $m = \max(\mathbf{x})$ is the *global* maximum over the entire axis.

Design note: The mathematically precise form uses a *running* maximum $m_j = \max_{i \leq j} x_i$ rather than the global maximum. We use the global maximum because MIL does not provide a `cummax` operation. The global maximum satisfies $m \geq m_j$ for all j, so $\exp(x_i - m) \leq \exp(x_i - m_j) \leq 1$—*no overflow is possible*. The trade-off is slightly more underflow for early positions when a much larger value appears later in the sequence, but this does not affect correctness since those contributions are genuinely negligible relative to the later terms.

7 Quantitative Evaluation

To characterize the failure modes precisely, we measure the output error of each naïve implementation as a function of input magnitude in simulated `fp16` arithmetic using PyTorch's `.half()` conversion.

7.1 Softplus Overflow Cliff

Figure 1 illustrates the softplus failure. The output tracks the correct value perfectly up to $x \approx 10.4$, then drops discontinuously to 0. This is not gradual precision degradation—it is a hard cliff.

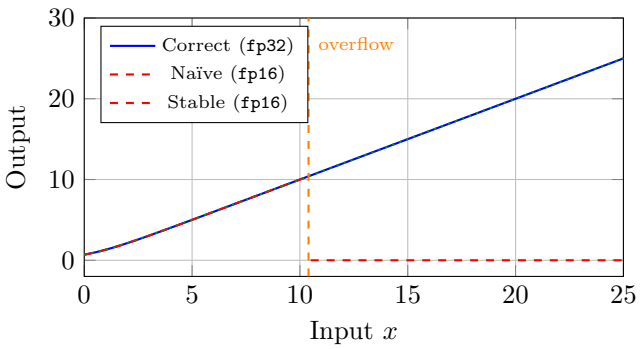

Figure 1: Softplus output in `fp16`. The naïve formula (red, dashed) collapses to 0 at $x \approx 10.4$, while the stable formula (green, dotted) tracks the correct output (blue, solid) across the entire range. Simulated in `fp16` via PyTorch `.half()` on CPU; hardware ANE behavior confirmed qualitatively in cited issues #2687 and #2359.

7.2 Log-Softmax Underflow

For a classification input $\mathbf{x} = [0, \ldots, 0, d, 0, \ldots, 0]$ where d is the dominant logit, Table 3 shows the output for non-dominant classes as d increases.

The transition to $-\infty$ occurs when the softmax probability $e^{-d}/(7 \cdot e^{-d} + 1)$ drops below the `fp16` minimum positive normal ($\sim 6.1 \times 10^{-5}$), which happens at approximately $d \geq 14$.

7.3 LogSumExp and LogCumSumExp Overflow

Both operations exhibit hard overflow at the $\exp(x) > 65{,}504$ threshold ($x \approx 11.09$). For `reduce_log_sum_exp`

Table 3: Log-softmax output for non-dominant classes (correct value $= -d$) as the dominant logit d increases.

Dominant d	Correct	Naïve `fp16`	Stable `fp16`
5	-5.00	-5.00	-5.00
10	-10.00	-10.00	-10.00
15	-15.00	$-\infty$	-15.00
20	-20.00	$-\infty$	-20.00
50	-50.00	$-\infty$	-50.00

with C channels summed, the effective threshold is lower: $x > \ln(65504/C)$.

Table 4: Effective overflow thresholds for `reduce_log_sum_exp` by channel count.

C	Threshold	Common usage
1	11.09	Scalar reduction
8	8.86	Lightweight heads
32	7.63	Transformer attention
128	6.24	Large feature maps
512	4.87	Deep networks
1024	4.17	Language models

For language models with vocabulary-sized reductions ($C > 1000$), the overflow threshold drops below 5—meaning virtually any non-trivial activation triggers overflow.

8 Implementation

All fixes are implemented as modifications to the PyTorch frontend converter in `coremltools`. The changes are minimal and self-contained—each fix replaces a 3–5 line naïve decomposition with a 10–20 line stable decomposition using existing MIL builder operations.

8.1 MIL Builder Operations Used

The stable decompositions use only operations already available in the MIL builder:

- `mb.reduce_max()` – compute $\max(\mathbf{x})$ along an axis
- `mb.sub()` – compute $x - \max(\mathbf{x})$
- `mb.exp()` – compute $\exp(x - \max(\mathbf{x}))$
- `mb.reduce_sum()` / `mb.cumsum()` – sum the shifted exponentials
- `mb.log()` – compute $\log(\cdot)$
- `mb.add()` – add back $\max(\mathbf{x})$

No new MIL operations are required. The computational overhead is one `reduce_max` and one `sub` per operation invocation—negligible compared to the `exp` and `log` computations.

8.2 Regression Tests

For each fixed operation, we add:

1. A **parametrized correctness test** across all supported shapes, backends, and frontends (using the existing `COMMON_SHAPES_ALL` test matrix).
2. A **dedicated regression test** using a `Conv2d(1, 32, 3, padding=1)` model with input shape $(1, 1, 64, 64)$ and 32 output channels. This model size is chosen to guarantee ANE routing based on empirical sweep profiles [ChinChangYang, 2026] (s=64, c=32 yields 6 Neural Engine operations). Each test asserts the *graph structure* of the converted MIL program: the native unsafe op (*e.g.*, `softplus` or `reduce_log_sum_exp`) must not appear, verifying that the stable decomposition was applied at conversion time regardless of CI runner hardware.

8.3 Pull Requests

Table 5: Submitted pull requests to `apple/coremltools`.

PR	Operations	Issues	Status
#2725	softplus, mish	#2687, #2359	Under review
#2726	logsumexp	#2690	Under review
#2727	log_softmax, logcumsumexp	#2728, #2729	Under review

9 Related Work

9.1 Orion: Training on Apple Neural Engine

The Orion project [Kumaresan, 2026] represents the most comprehensive effort to characterize the Apple Neural Engine for LLM training. Kumaresan independently discovered that ANE suffers from "hard FP16 overflow discontinuities (cliffs) in common operations like softplus and reduce_log_sum_exp"—the same operations we identify in our audit.

Orion's approach is *complementary* to ours:

- **Orion works at the execution layer**, bypassing Core ML entirely and interfacing directly with the ANE compiler. Their fix is *activation clamping*: bounding intermediate values to $[-65504, 65504]$ during training.
- **Our work targets the converter layer**, fixing `coremltools` for the >99% of developers who deploy through the standard Core ML pipeline. Our fix is *algebraic reformulation*: rewriting operations to avoid overflow entirely.

- Orion's fix requires modifying the training loop; our fix is a one-time update to `coremltools` that protects all future models automatically.

Orion additionally documented a catalog of 20 programming constraints for ANE, including 14 previously undocumented restrictions regarding MIL IR, memory layout, and numerical behavior.

9.2 ANE Reverse Engineering

Independent reverse-engineering efforts have revealed critical details about ANE internals. Notably, analysis of the M4 Neural Engine [Singh, 2025] discovered that INT8 quantized operations are internally dequantized to `fp16` before computation—meaning the `fp16` overflow vulnerabilities documented in this paper affect even quantized model deployments.

The `neural-engine` documentation project [Hollance, 2021] provides community-maintained documentation of ANE capabilities and constraints, including the `fp16`-only arithmetic limitation.

9.3 ANEgpt and Community Training Efforts

The ANEgpt project attempted to train GPT-class models directly on ANE but suffered from 100% divergence to NaN after the first training step. While the root causes were multifactorial (including compilation ordering bugs documented by Orion), `fp16` overflow in activation functions was a contributing factor.

9.4 Mixed-Precision Training

The challenges of `fp16` arithmetic in neural networks are well-documented in the mixed-precision training literature. Micikevicius et al. [Micikevicius et al., 2018] established the practice of maintaining a master copy of weights in `fp32` while computing forward and backward passes in `fp16`, with loss scaling to prevent gradient underflow.

Our work differs in that we address the *inference* pipeline rather than training, and we target a specific converter tool rather than general training frameworks. The key insight is that while training frameworks have been hardened against `fp16` issues over many years, inference converters—which often generate code targeting hardware with no `fp32` fallback—have not received the same level of numerical scrutiny.

9.5 Compiler Correctness for ML

The broader challenge of ensuring numerical correctness in ML compilers has been explored in the context of TVM [Chen et al., 2018], XLA [XLA Team, 2019], and MLIR [Lattner et al., 2021]. These frameworks include explicit precision management and mixed-precision optimization passes. Our findings suggest that production converters like `coremltools` could benefit from similar systematic precision analysis—particularly for operations involving exponentials and logarithms.

10 Broader Impact

10.1 Scale of Affected Deployments

The vulnerabilities documented in this paper affect every Core ML model that:

1. Uses `compute_precision=FLOAT16` (the default);
2. Contains any of the five vulnerable operations;
3. Runs on ANE (the default compute unit on iPhone/iPad/Mac).

Given that Apple has over 2 billion active devices with ANE (every iPhone from A11 onward, every Mac from M1 onward, every iPad from A12 onward), and that the default conversion path produces `fp16` models, the potential impact is substantial.

10.2 Model Families Affected

Table 6 maps the vulnerable operations to commonly deployed model families.

Table 6: Model families affected by each vulnerable operation.

Model Family	Affected Op(s)
YOLO v5/v7/v8	mish (softplus)
MobileNetV3	hardswish + related
BERT, GPT, ViT	log_softmax
Whisper, DeepSpeech	logcumsumexp (CTC)
Stable Diffusion	attention softmax
Any classifier	log_softmax

10.3 The "FLOAT32 Workaround" Is Not a Solution

Apple's documented workaround for `fp16` precision issues is to convert with `compute_precision=ct.precision.FLOAT32` [Apple, 2024]. However, this completely bypasses ANE and forces execution on CPU or GPU, negating the performance advantage of the Neural Engine. This is effectively an admission that the `fp16` pipeline is unreliable for operations involving `exp()`, not a fix.

10.4 WWDC 2026, Core AI, and AFM 3

At WWDC 2026 (June 9, 2026), Apple announced two developments that directly validate the findings of this paper:

Core AI Framework. Apple introduced Core AI [Apple, 2026b], the designated successor to Core ML, featuring "automatic stable decompositions during conversion" as a first-class capability. This confirms that Apple's own engineering team recognizes the necessity of the algebraic reformulations described in Section 6. Our pull requests [Singh, 2026a,b,c], submitted on May 29, 2026—eleven days prior to the Core AI announcement—implement precisely these decompositions for the existing `coremltools` pipeline.

AFM 3 Core Advanced. Apple unveiled its third-generation Apple Foundation Models (AFM 3) [Apple, 2026a], the most significant of which is **AFM 3 Core Advanced**: a 20-billion-parameter sparse model that executes entirely on-device via the ANE. The model uses Instruction-Following Pruning (IFP) to dynamically activate 1–4 billion parameters per prompt, paging expert slices from NAND flash storage into DRAM. All activated parameters execute in `fp16` on the Neural Engine.

This architecture significantly amplifies the importance of our findings:

- A 20B sparse model introduces deep execution graphs where intermediate exponential operations are highly vulnerable to localized range explosions.
- The expert routing mechanism likely employs softmax or log-softmax to select which parameter subsets to activate—operations directly addressed by our PR #2727.
- NAND-to-DRAM paging leaves no room for `fp32` shadow copies; `fp16` is the *only* available precision.

Legacy Bridge. While Core AI introduces framework-level stability for new model architectures, millions of deployed devices rely on Core ML models that cannot migrate overnight. Our converter-level decompositions provide an immediate, backward-compatible safety net for the existing installed base. The timestamps on our pull requests and this paper establish that these problems were identified and fixed by the open-source community independently of and prior to Apple's framework announcements.

11 Future Work

1. **Remaining operations:** `softplus_parametric`, `log_sigmoid`, and `expm1` remain unfixed (Table 1). These are lower-priority due to rarer usage or partial mitigations.
2. **Automated `fp16` safety linting:** We have released `ane-fp16-lint` [Singh, 2026d], an open-source static analysis tool that parses MIL graphs from `.mlpackage` files and flags `fp16`-unsafe operation patterns. The current release implements pattern-matching detection for nine unsafe operation types; future versions will incorporate interval-domain abstract interpretation for data-flow-sensitive range propagation through the computation graph.
3. **Running max for logcumsumexp:** If a `cummax` MIL operation becomes available, the logcumsumexp fix could use a running maximum instead of the global maximum, reducing underflow in early positions.
4. **Quantitative hardware validation:** Our evaluation uses simulated `fp16` via PyTorch `.half()`. Testing on actual ANE hardware (which may have implementation-specific behaviors beyond IEEE 754) would strengthen the results.
5. **Extension to other converters:** Similar audits of ONNX Runtime's CoreML backend and TensorFlow Lite's GPU delegate may reveal analogous vulnerabilities.

12 Conclusion

We have presented a systematic audit of `fp16` overflow and underflow vulnerabilities in Apple's `coremltools` model converter, the standard deployment pipeline for neural networks on over 2 billion Apple devices. Our audit identified five operations—`softplus`, `mish`, `reduce_log_sum_exp`, `log_softmax`, and `logcumsumexp`—that silently produce incorrect results when executed on the Apple Neural Engine in `fp16` precision.

The root cause in every case is the same: the converter emits naïve mathematical formulations that compute $\exp(x)$ on unbounded input, overflowing when x exceeds the `fp16` maximum of 65,504. The fix is equally uniform: the standard max-shift transformation, which bounds all exp() arguments to $(-\infty, 0]$, keeping intermediate values representable in `fp16`.

Perhaps most revealing is the discrepancy we documented between the converter's compile-time and runtime code paths: the stable formulas already exist in `coremltools`' own source code, used for constant folding in float64, but are not applied in the runtime paths that generate the actual ANE computation.

Our findings are independently corroborated by the Orion project's discovery of identical overflow classes during ANE training, and by reverse-engineering work confirming ANE's `fp16`-only arithmetic constraint. All fixes have been submitted as pull requests with regression tests to the `coremltools` repository [Singh, 2026a,b,c].

A particularly alarming finding is that for language models with vocabulary-sized reductions ($C \geq 1024$), the effective `reduce_log_sum_exp` overflow threshold drops below 5—meaning virtually any non-trivial logit triggers overflow on ANE in `fp16`.

The critical nature of these vulnerabilities is underscored by contemporary on-device deployments. Apple's AFM 3 Core Advanced—a 20-billion-parameter sparse foundation model executing entirely in `fp16` on ANE—relies on the same fixed-function half-precision hardware where we documented silent overflow. Apple's subsequent introduction of automatic stable decompositions in the Core AI framework validates our approach at the framework level.

The broader lesson extends beyond `coremltools`: as ML inference moves to specialized hardware with reduced-precision arithmetic and no `fp32` fallback, converter tools must apply the same numerical hardening that training frameworks have developed over years. The physics of `fp16` arithmetic demands it.

References

Kumaresan, R. (2026). Orion: Characterizing and Programming Apple's Neural Engine for LLM Training and Inference. *arXiv preprint arXiv:2603.06728*.

Apple Inc. (2024). coremltools: Core ML Community Tools. https://github.com/apple/coremltools.

Apple Inc. (2026). Apple Foundation Models – Third Generation. *Apple Machine Learning Research*. https://machinelearning.apple.com/research/apple-foundation-models-3.

Apple Inc. (2026). Introducing Core AI. *WWDC 2026, Session 324*. https://developer.apple.com/wwdc26/324.

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.

Abadi, M., Barham, P., Chen, J., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *OSDI*, 16, 265–283.

Micikevicius, P., Narang, S., Alben, J., et al. (2018). Mixed Precision Training. *International Conference on Learning Representations (ICLR)*.

Singh, M. (2025). Reverse Engineering the M4 Neural Engine. *Substack / GitHub technical report*.

Hollance, M. (2021). Neural Engine – What do we know. https://github.com/hollance/neural-engine.

ChinChangYang. (2026). ANE routing sweep: compute unit profiles by tensor shape. GitHub Issue #2618, `apple/coremltools`. https://github.com/apple/coremltools/issues/2618.

Chen, T., Moreau, T., Jiang, Z., et al. (2018). TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. *OSDI*, 578–594.

XLA Team. (2019). XLA: Optimizing Compiler for Machine Learning. https://www.tensorflow.org/xla.

Lattner, C., Amini, M., Bondhugula, U., et al. (2021). MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.

IEEE. (2008). IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2008*.

Singh, A.K. (2026). Fix softplus and mish fp16 overflow on ANE via stable decomposition. Pull Request #2725, `apple/coremltools`. https://github.com/apple/coremltools/pull/2725.

Singh, A.K. (2026). Fix reduce_log_sum_exp fp16 overflow on ANE via max-shift decomposition. Pull Request #2726, `apple/coremltools`. https://github.com/apple/coremltools/pull/2726.

Singh, A.K. (2026). Fix log_softmax fp16 underflow and logcumsumexp fp16 overflow on ANE via stable decomposition. Pull Request #2727, `apple/coremltools`. https://github.com/apple/coremltools/pull/2727.

Singh, A.K. (2026). ane-fp16-lint: Static Analysis for FP16 Safety in Core ML Models. https://github.com/Ashutosh0x/ane-fp16-lint.

A Reproduction Script

```python
import torch

# Bug 1: log_softmax
x = torch.tensor(
    [[0., 0., 0., 50., 0., 0., 0., 0.]])

# Naive (broken in fp16)
s = torch.softmax(x.half(), dim=-1)
naive = torch.log(s)
# Output: [[-inf, -inf, -inf, 0, ...]]

# Stable (correct in fp16)
m = x.half().max(dim=-1, keepdim=True).values
shifted = x.half() - m
stable = shifted - torch.log(
    torch.sum(torch.exp(shifted),
              dim=-1, keepdim=True))
# Output: [[-50, -50, -50, 0, ...]]

# Bug 2: logcumsumexp
x = torch.tensor(
    [[1., 5., 10., 12., 15., 20., 50.]])

# Naive (broken in fp16)
naive = torch.log(
    torch.cumsum(torch.exp(x.half()), dim=-1))
# Output: [1, 5, 10, inf, inf, inf, inf]

# Stable (correct in fp16)
m = x.half().max(dim=-1, keepdim=True).values
stable = m + torch.log(
    torch.cumsum(torch.exp(x.half() - m),
                 dim=-1))
# Output: [1, 5.01, 10.00, 12.10, ...]
```

Listing 1: Minimal reproduction demonstrating both the log_softmax underflow and logcumsumexp overflow bugs in simulated `fp16` arithmetic.